Inter-agent communication patterns for multi-agent AI systems

The foundation of effective multi-agent AI coordination rests on three interconnected systems: **structured messaging** with defined lifecycles, **intent/plan broadcasting** via shared state patterns, and **coordination protocols** that prevent conflicts while maximizing parallelism. Modern frameworks like CrewAI, AutoGen, and LangGraph each implement distinct approaches, but all converge on common patterns: message queues with priority handling, blackboard-style shared state, and FIPA-derived negotiation protocols. For MCP tool design, the most practical path combines pub/sub messaging with event-driven plan updates and capability-based routing.

Message queue patterns enable flexible agent communication

Agent-to-agent messaging in multi-agent systems follows three primary patterns, each suited to different coordination scenarios. **Work queues** provide direct one-to-one task distribution where messages reach exactly one consumer from a pool—ideal for load-balanced task execution. **Publish/subscribe** enables fan-out communication where agents subscribe to topics or event types, decoupling senders from receivers and supporting system-wide announcements. **Request/reply** creates synchronous-style exchanges using correlation IDs to match responses to requests, essential for tool calls and capability queries. sparkco ai

The technology choice significantly impacts system behavior. Redis Pub/Sub delivers **59,000+ messages per second** GitHub but offers no persistence—suitable for real-time signaling. RabbitMQ provides durable queues with **4,000-10,000 messages per second** and reliable delivery guarantees. EDUCBA For large-scale systems exceeding 100 agents, Kafka's distributed log architecture handles **100,000+ messages per second** with built-in event sourcing. Most multi-agent implementations benefit from asynchronous messaging with at-least-once delivery, requiring idempotent message handlers to gracefully manage duplicates. sparkco ai

The recommended message schema for short coordination messages (~200 characters) includes:

```json
```

```
{
  "message_id": "msg_uuid",
  "sender_id": "agent_researcher_01",
  "recipient_id": "agent_coordinator",
  "message_type": "request",
  "content": {"text": "Research complete", "payload": {}},
  "timestamp": "2025-01-15T10:30:00Z",
  "priority": "high",
  "thread_id": "thread_research_task",
  "reply_to": "msg_previous",
  "status": "sent"
}
```


Message types should distinguish between **request**, **response**, **notification**, **broadcast**, and **acknowledgment** to enable proper routing and handler selection. (Medium) Priority levels (urgent, high, normal, low) allow critical coordination messages to preempt routine updates—implementing this requires priority queues at the broker level, where RabbitMQ supports up to 255 priority levels per queue.

Message lifecycle management ensures reliable delivery

Messages progress through a defined state machine: **draft → sent → delivered → read → archived**, with branches for expiration, retry, and failure. Delivery confirmation follows three patterns depending on criticality. At-least-once delivery with retransmission until acknowledgment (To Infinite Scale) suits mission-critical agent tasks but requires idempotent handlers. At-most-once (fire-and-forget) works for non-critical status updates. (sparkco ai) Exactly-once delivery—achieved through message ID deduplication with TTL-bounded storage—provides the strongest guarantees but adds complexity. (To Infinite Scale)

Retry logic should implement exponential backoff: (delay = base_delay × 2^attempt) with a maximum cap (typically 30 seconds) and a retry limit of 3-5 attempts. Messages failing all retries route to a dead letter queue for manual inspection or alternative handling. (DockYard) **Time-to-live (TTL)** prevents queue bloat—coordination messages typically expire after 60-3600 seconds, with expired messages either archived or routed to dead letter exchanges. (RabbitMQ)

Threading enables conversation chains through either **hierarchical paths** ((thread_root/reply_1/reply_2)) that support single-query rendering, or **parent references** that require joins for deep threads but simplify schema design. (Redgate Software) For conversations exceeding 50 messages, storing checkpoint summaries preserves context without requiring full history retrieval. (GitHub)

Multi-agent frameworks implement distinct shared state patterns

Modern frameworks take fundamentally different approaches to agent coordination. **CrewAI** uses role-based task flow where agents announce work through explicit Task objects with (description) and

expected_output fields. CrewAI Delegation occurs via tool-mediated handoffs (DelegateWorkToCoworker, AskQuestionToCoworker) with allowed_agents controlling delegation targets. State sharing happens through task context parameters that pass results between dependent tasks. CrewAI

AutoGen centers on conversational coordination through a shared message pool. Group Chat Manager orchestrates turn-taking using LLM-based selection, round-robin, or finite state machine transitions. arXiv GitHub Sequential chats enable multi-step workflows with carry-over mechanisms injecting summaries from previous conversations as context. GitHub

LangGraph implements graph-based state management with TypedDict schemas serving as central contracts. Nodes communicate by updating shared state rather than direct messages, with reducers controlling whether updates override or accumulate. The Command pattern allows explicit routing

(Command(goto="next_node")), while checkpointers persist state for resumability. Langchain

MetaGPT employs standardized operating procedures (SOPs) encoding workflows as prompt sequences. arXiv Communication follows a publish-subscribe pattern where agents publish structured messages to a global pool and subscribe via _watch([ActionType]) to relevant message types—no direct agent-to-agent communication exists.

Framework	State Pattern	Delegation	Intent Visibility
CrewAI	Task Context	Tool-based	Task outputs
AutoGen	Message Pool	Group Chat	Messages
LangGraph	TypedDict State	Graph Edges	State fields
MetaGPT	Global Message Pool	SOP-based	Structured artifacts

Blackboard architecture provides shared problem-solving space

The blackboard pattern—where agents read from and write to a shared knowledge base while a control component coordinates activation—proves particularly effective for ill-defined problems requiring incremental solution building. Confluent Knowledge sources (agents) contribute based on expertise, with no direct inter-agent communication required.

Modern implementations adapt this for LLM agents through **bMAS** (blackboard-based LLM Multi-Agent Systems) patterns. Public shared space stores the problem specification, partial solutions, and agent contributions. Private spaces maintain per-agent context. Special-purpose agents include a **Conflict-Resolver** detecting contradictions and facilitating discussion, plus a **Cleaner** removing redundant messages.

Event-driven blackboards using Kafka topics enable real-time updates: agents publish structured events with agent_id, action type, payload, and timestamp, while subscribing agents react to relevant changes. InfoWorld +2 This approach scales better than polling but requires careful state consistency management.

Intent declaration enables proactive conflict prevention

Agents declaring intentions before acting allows systems to detect conflicts proactively. An effective intent schema captures:

```typescript
interface AgentIntent {
  intent_id: string;
  agent_id: string;
  action: string;          // What agent will do
  target: string;          // Object of action
  rationale: string;       // Why this action
  expected_outcome: string;
  visibility: "public" | "group" | "private";
  priority: number;
  dependencies: string[];  // Required prior intents
  timestamp: Date;
  expires_at?: Date;
}
```

Conflict detection compares active intents for overlapping targets with incompatible actions. When Agent A declares intent to modify resource X while Agent B has a pending intent for the same resource, the system triggers resolution before either agent acts. Resolution strategies include **priority-based** (higher-priority agent wins), **first-come-first-served** (based on declaration timestamp), or **negotiation-based** using the FIPA Contract Net Protocol.

Plan visibility requires hierarchical representation: **goal → sub-goals → tasks → current_task**. Progress tracking calculates completion percentage from task states, while version tracking enables auditability and rollback. Event-driven plan updates (rather than polling) provide real-time coordination with delta updates for efficiency and full synchronization when versions diverge.

Coordination protocols derive from distributed systems theory

Agent status management centers on a five-state model: **active** (executing task), **idle** (available for work), **blocked** (waiting for external resource), **waiting** (awaiting another agent), and **error** (encountered recoverable problem). Heartbeat patterns detect liveness—push-based heartbeats every 30-60 seconds with

a miss threshold of 3-4 consecutive failures trigger agent failure handling.

The **FIPA Contract Net Protocol** provides the standard for task allocation. A manager broadcasts Call-for-Proposals, contractors respond with proposals including estimated completion, confidence scores, and resource requirements, the manager selects winner(s), and the contractor executes and reports. Wikipedia Message performatives include cfp, propose, accept-proposal, reject-proposal, inform-done, and failure. Fipa

Resource locking prevents conflicts through lock types ranging from Null (interest indicator) through Concurrent Read/Write, Protected Read/Write, to Exclusive access. DEV Community Wikipedia **Lease-based locking** with automatic expiration prevents deadlocks—locks include expires_at timestamps and release automatically when agents fail. Deadlock prevention algorithms like Wait-Die (older transactions wait, younger abort) or timeout-based release provide additional safety.

For multi-step operations, the **Saga pattern** manages distributed transactions. Orchestration-based sagas use a central coordinator maintaining a task ledger with compensating actions for rollback. Choreography-based sagas decentralize control through event publication—each agent knows only its local transaction and the next event to emit.

Database schema design for agent messaging and plans

The messaging system requires three core tables. The **messages** table stores sender_id, recipient_id, message_type, content (JSONB), status, priority, thread_id, reply_to, metadata, and timestamps for created_at, delivered_at, and read_at. GeeksforGeeks Critical indexes cover (recipient_id, status, created_at DESC) for inbox queries, (sender_id, created_at DESC) for outbox, (thread_id, created_at ASC) for conversation retrieval, and a partial index on status for unread message counts.

sql

```sql
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sender_id VARCHAR(100) NOT NULL,
    recipient_id VARCHAR(100) NOT NULL,
    message_type VARCHAR(20) NOT NULL,
    content JSONB NOT NULL,
    status VARCHAR(20) DEFAULT 'sent',
    priority VARCHAR(10) DEFAULT 'normal',
    thread_id UUID,
    reply_to UUID REFERENCES messages(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    delivered_at TIMESTAMP WITH TIME ZONE,
    read_at TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_messages_inbox ON messages(recipient_id, status, created_at DESC);
CREATE INDEX idx_messages_unread ON messages(recipient_id) WHERE status IN ('sent', 'delivered');
```

The **agent_plans** table tracks plan_id, agent_id, plan_version, goal_description, plan_status, current_task_id (foreign key to plan_tasks), total_tasks, completed_tasks, and timestamps. A generated column calculates `progress_percentage` automatically. The **plan_tasks** table enables hierarchy through parent_task_id self-reference, with task_order and depth_level supporting traversal.

sql

```sql
CREATE TABLE agent_plans (
    plan_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id VARCHAR(100) NOT NULL,
    plan_version INTEGER NOT NULL DEFAULT 1,
    goal_description TEXT NOT NULL,
    plan_status VARCHAR(50) NOT NULL DEFAULT 'active',
    current_task_id UUID,
    total_tasks INTEGER DEFAULT 0,
    completed_tasks INTEGER DEFAULT 0,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE agent_intents (
    intent_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id VARCHAR(100) NOT NULL,
    intent_type VARCHAR(100) NOT NULL,
    action VARCHAR(255) NOT NULL,
    target VARCHAR(255),
    rationale TEXT,
    visibility VARCHAR(50) DEFAULT 'public',
    priority INTEGER DEFAULT 5,
    status VARCHAR(50) DEFAULT 'declared',
    declared_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    expires_at TIMESTAMP WITH TIME ZONE
);
```

```
);

CREATE INDEX idx_intents_active ON agent_intents(status, expires_at)
  WHERE status IN ('declared', 'active');
CREATE INDEX idx_intents_target ON agent_intents(target);
```

For coordination, an **agent_status** table tracks agent_id, status enum, current_task_id, capabilities (JSONB), and last_heartbeat. A **resource_locks** table records resource_id, holder_agent_id, lock_type, acquired_at, and expires_at with a unique constraint on (resource_id, holder_agent_id). Event sourcing through a **coordination_events** table captures event_type, timestamp, initiating_agent_id, participating_agents array, and outcome for complete audit trails.

Time-based partitioning by month handles high-volume messaging, while archival strategies move messages older than 90 days with status 'read' or 'archived' to separate archive tables. Cleanup jobs should run during low-traffic periods to minimize lock contention.

Conclusion: practical patterns for MCP tool design

Effective MCP tools for agent communication should implement **three-tier architecture**: a messaging layer using pub/sub with priority queues and defined message lifecycles, a shared state layer following blackboard or graph-based patterns with intent declaration and conflict detection, and a coordination layer providing status management, capability-based routing, and FIPA-compliant negotiation protocols.

Key implementation priorities include message schemas with required threading support (thread_id, reply_to) and **200-character content limits** for coordination messages, intent declaration with automatic conflict detection before agent action, event-driven plan updates with version tracking and delta synchronization, lease-based resource locking with automatic expiration to prevent deadlocks, and comprehensive event sourcing for debugging and audit.

The most robust pattern combines **orchestrator-worker coordination** (where a lead agent manages specialized subagents) with **capability-based routing** (directing tasks to agents with matching tools and expertise). Anthropic Langchain This approach, validated by Anthropic's multi-agent research system, trades approximately **15x token usage** for dramatically improved output quality on complex tasks while enabling parallel execution with up to 90% latency reduction. anthropic